

Сравнительная таблица: Pictiq vs аналоги

Критерий	Point It	Kwikpoint	ICOON
Создатель/ происхождение	Дитер Граф, архитектор- путешественник, идея из поездки по Индии 1976 г. (Welt)	Алан Стиллман, велопутешественник, идея из поездки по Европе конца 1980-х (Works That Work)	Гося Варринк, дизайнер соц. коммуникации, Берлин (Society of Signs)
Год запуска	1992 (16 лет разработки)	1991	2007
Визуальный носитель смысла	Фотографии (осознанный отказ от рисунков из-за искажения масштаба)	Иллюстрированные иконки от команды художников	Смесь символов, пиктограмм и фото
Размер словаря	1300 предметов (graf-editions.de)	~1000 символов (Wikipedia)	1200–3000+ в зависимости от издания (icoon-book.com)
Грамматика/ композиция	Нет — чистое указание на картинку	Нет	Нет (частично — "communicator" сочетает символ+вопрос/ ответ)
Формат выпуска	Карманная книга 9×13 см, 72 стр.	Складные карты/буклеты	Книга + приложение (iOS/Android)
Цена	€7,90	Не раскрыта публично	€5,90–9,95 в зависимости от издания (icoon-book.com)

Критерий	Point It	Kwikpoint	ICOON
Канал дистрибуции	Книжные магазины, аэропорты	Прямые продажи + госконтракты (военные/мед.)	Собственный интернет-магазин + благотворительные каналы
Целевая аудитория	Массовый турист	Турист → военные/госслужбы → медики (эволюция!)	Турист → беженцы/гуманитарный сектор
Бизнес-модель	Продажа тиража (роялти автору-издателю)	B2B/госконтракты + розница	Розница + гранты/краудфандинг для гуманитарной версии
Управление/стандартизация	Нет формального органа, всё держит сам автор	Нет, закрытая разработка	Нет формального органа
Цифровое присутствие	Принципиально офлайн (сопротивлялся цифровизации) (Welt)	Минимальное	Полноценное приложение
Ключевое УТП	"Первый и самый продаваемый" — сила бренда и охвата (2,5 млн копий)	Многовертикальность одного движка (турист→военные→медики)	Гибкость: словарь превращается в гуманитарный инструмент

Главный вывод из таблицы: ни один аналог не предлагает **грамматику** — они все словари. Это единственная категория, где Pictiq принципиально впереди, и именно её нужно подчёркивать в позиционировании и на обложке.

Ситуационные продукты и наборы иконок

Использую уже существующий лексикон Pictiq (52 иконки + категории money/needs/places/movement/items/drinks/social) плюс кандидатов из вашего же бэклога

(person, hot/cold, clothing и т.д.), там где не хватает.

1. Бар / ночная жизнь

У вас уже есть заготовка (layouts/lighter/nightlife) — вот как её можно расширить в полноценный кит.

- **Форм-фактор:** обёртка на зажигалку (уже есть), подставка под бокал (beer mat), значок/пин на сумку, гардеробный жетон с QR на цифровой словарь.
- **Иконки:** drink_beer + qty_1/qty_2/qty_5 (заказ количества), money_card / money_coins + punct_question ("можно картой?"), need_toilet + punct_question, logic_no (отказ от навязчивого угощения/флирта), love_heart (социальный интерес), item_cigarette, place_disco, move_taxi (такси домой), time ("во сколько закрытие?").
- **Композиции-фразы:** drink_beer + qty_2 + punct_question ("два пива, можно?"), logic_no + love_heart (вежливый отказ), move_taxi + punct_question ("такси есть?").
- **Идея нового тайла:** индикатор "уже расплатился/чек, пожалуйста" (bill/check) — частый и пока не покрытый кейс.

2. Шоппинг / ритейл

- **Форм-фактор:** карточка-кошелёк (уже есть шаблон wallet-card), нашивка на сумку-шоппер, кассовый лайтбокс-стикер для витрины ("здесь понимают Pictiq" — интересный B2B-мерч для магазинов, работающих с туристами).
- **Иконки:** money_card, money_coins, money_atm_bank, qty_plus / qty_minus (скидка/размер побольше-поменьше), logic_yes / logic_no (можно примерить / нет, не подходит), place_fashion_shopping, place_shop, punct_question.
- **Недостающий тайл:** одежда/размер (уже в вашем бэклоге как "clothing — lexical-tile candidate") — стоит приоритезировать именно под этот продукт.
- **Композиции:** place_fashion_shopping + qty_minus + punct_question ("есть скидка?"), money_card + logic_no + money_coins ("картой нельзя, только наличные").

3. Ресторан / заказ еды

- **Форм-фактор:** настольная карточка-подставка (table tent) в самом ресторане, либо личная карманная карта туриста для диетических ограничений.
- **Иконки:** need_food, need_water, qty_1..qty_5, money_coins / money_card, logic_no (аллергия/не хочу этот ингредиент), punct_exclaim (остро/осторожно), time ("сколько ждать?").

- **Недостающие тайлы из бэклога, критичные именно здесь:** produce (фрукты/овощи), bakery/bread — уже отмечены как кандидаты, но для ресторанного кита они первостепенны наравне с "аллергия" (сейчас выражается только через универсальный logic_no, что менее однозначно, чем отдельный значок "аллерген").
- **Композиция:** need_food + logic_no + qty_1 (аналог "без одного из ингредиентов" — компаунд из спецификации).

4. Аэропорт / транзит

Здесь у вас уже готов luggage-tag и travel-transit профиль — расширение:

- **Форм-фактор:** держатель посадочного талона, стикер на чемодан, лента для сумки.
- **Иконки:** move_taxi, move_public, move_boat, move_feet, place_airport, place_hotel, time, punct_question.
- **Композиция:** place_hotel + move_taxi + punct_question ("такси до отеля?").

5. Экстренная помощь / безопасность

Прямая параллель с Kwikpoint Medical Translator — вероятно, самая коммерчески защищённая ниша (люди платят больше за товары "на всякий случай").

- **Форм-фактор:** карточка в бумажник/паспорт, брелок, вставка в чехол телефона, браслет для детей/пожилых с медицинскими особенностями (аналог MedicAlert).
- **Иконки:** safety_medical, safety_police, punct_exclaim, need_water, need_toilet, logic_no (аллергия на лекарство).
- **Недостающий тайл:** нейтральный "человек/участник" (уже STRONG CANDIDATE в вашем бэклоге) — именно для этого продукта он особенно нужен ("этот человек нуждается в помощи").
- **Композиция:** safety_medical + punct_exclaim ("срочно, медицинская помощь!").

6. Отель / проживание

- **Форм-фактор:** дверная табличка (не беспокоить / уберите номер), вставка в кардхолдер ключа-карты.
- **Иконки:** place_hotel, comm_wifi, power_plug, need_water, time, money_card, logic_yes / logic_no.
- **Композиция:** comm_wifi + punct_question ("какой пароль от wifi?"), logic_no + time ("не беспокоить до [времени]" — как только появится параметрический тайл времени).

7. Активный отдых / природа / поход

- **Форм-фактор:** стикер на бутылку воды, нашивка на рюкзак, браслет.
- **Иконки:** `nature_flower`, `move_feet`, `place_landmark_park`, `safety_medical`, `need_water`.
- **Недостающий тайл:** `sun/day` и `hot/cold` (уже в бэклоге как HIGH-UTILITY CANDIDATE) — прямое попадание именно в этот продукт (тепловой удар/переохлаждение — реальный риск в походе).

8. "Локальные законы" / нелегальный статус товаров

Интересная ниша именно потому, что у вас уже есть готовые иконки `item_cannabis`, `item_cigarette`, `item_condom` — редкий набор, который вообще мало у кого есть.

- **Форм-фактор:** маленькая карта-брелок "что легально здесь?" — по одной на страну/город (отличный контекст-пак товар, например для Амстердама, где легальность каннабиса — реальный вопрос туристов).
- **Иконки:** `item_cannabis` / `item_cigarette` / `item_condom` + `logic_yes` / `logic_no` в конкретном контекст-паке страны.
- **Композиция:** `item_cannabis` + `logic_yes` (легально) / `item_cannabis` + `logic_no` (запрещено) — прямая демонстрация вашей грамматики отрицания на практическом и слегка провокационном кейсе, что само по себе хороший маркетинговый крючок.

9. Цифровой кочевник / коворкинг

- **Форм-фактор:** стикер на ноутбук, табличка на кафе-столик ("здесь есть розетка и wifi для работы").
- **Иконки:** `comm_wifi`, `power_plug`, `tech_ai`, `money_card`, `place_shop`.
- **Композиция:** `comm_wifi` + `power_plug` + `punct_question` ("есть вайфай и розетка?").

10. Аренда авто / дорога

- **Форм-фактор:** стикер на приборную панель, брелок для ключей.
- **Иконки:** `move_car`, `place_gas`, `money_card`, `safety_police` (проверка документов на посту), `punct_question`.

11. Семейный туризм / "если ребёнок потерялся"

Отдельная и эмоционально сильная ниша (аналог браслетов "if found").

- **Форм-фактор:** браслет/нашивка на рюкзак ребёнка с QR на контакты + иконный ряд.
- **Иконки:** `safety_police`, `place_hotel` (куда вернуть), `need_toilet`, `love_heart`, кандидат "person" из бэклога.

Общий паттерн упаковки

Каждый из этих китов может использовать уже готовый у вас generator-pipeline (`tools/make_wallet_card.py` , `make_lighter.py` , `make_luggage_tag.py` и т.д.) — то есть с точки зрения производства это не 11 новых проектов, а 11 новых **content-профилей** (`layouts/profiles/*.json`) поверх уже существующей инфраструктуры рендеринга. Самая низкая цена входа — начать с бара (уже есть черновик) и локальных законов (уникальное УТП благодаря готовым `item_*` иконкам), а самая высокая коммерческая защищённость — у экстренной/медицинской карточки, где даже минимальный набор из 5–6 иконок уже создаёт продаваемый продукт по аналогии с Kwikpoint Medical Translator.

Pictiq и ИИ: где реально есть потенциал

Смотрю на это не как на общие рассуждения "ИИ может всё", а конкретно исходя из того, что уже заложено в вашей спецификации: в `PROTOCOL.md` уже есть раздел "Notes for computer vision / parsing" (детект границ тайла, различение line vs grid группировки, восстановление порядка токенов) — то есть машинная читаемость была в уме с самого начала, просто не реализована. Это хорошая стартовая точка.

Почему протокол структурно уже "AI-ready"

- **Закрытый, версионированный словарь + формальная грамматика** (MUST/SHOULD/MAY, decision tree) — это по сути мини-DSL (domain-specific language), а не свободная иконография.
- **JSON-реестр** (`icon-index.json` , паки, i18n) уже структурирован как машиночитаемая онтология с id, алиасами, тегами и примерами использования.
- **Монохромный canonical SVG без `<text>`** — это чистый геометрический сигнал, идеальный для CV-классификации без шума шрифтов/цвета.
- **Explicit rules про embodied vs standalone** — уже формализует, когда смысл переносится в контекст, а когда должен быть закодирован явно, что прямо соответствует тому, как проектируют промпты и системы с ограниченным контекстом.

Направление 1 — компьютерное зрение: распознавание тайлов

Здесь стоит быть честным: современные vision-language модели уже неплохо справляются с zero-shot интерпретацией простых пиктограмм без специального дообучения — но исследования также показывают, что это распознавание не такое надёжное, как кажется, особенно для составных/абстрактных диаграмм. Свежая работа **Visual Iconicity Challenge** отдельно тестирует VLM именно на способности связывать иконическую форму со значением — это готовая методологическая рамка, по которой можно сделать собственный бенчмарк для Pictiq. [\[openreview\]](#)[\[arxiv\]](#)[\[openreview\]](#)

Практическая идея: не "научить ИИ видеть", а **создать открытый бенчмарк** "насколько GPT-4V/Gemini/Claude понимают тайлы Pictiq zero-shot vs с подсказанным лексиконом" — это дешёвый, но заметный вклад в исследовательское сообщество (PR-эффект для проекта, плюс реальные данные вместо интуиции). У вас уже есть методология визуального QA (canonical → small → grid → layout из ICON_SPEC.md) — она почти без изменений переносится на машинное тестирование.

Отдельная и более прикладная идея: тайл как **гибридный fiducial-маркер** — в отличие от QR-кода, который нечитаем человеком, напечатанный/показанный на экране Pictiq-тайл одновременно понятен человеку и легко детектируется простым классификатором (рамка + монохромная форма). Это открывает нишу "маркеры двойного назначения" для вывесок, упаковки, инструкций — **機**ина и человек читают один и тот же знак.

Направление 2 — промптинг и структурированная генерация

Закрытый словарь идеально ложится на **constrained decoding** — методику, которой сейчас активно занимается индустрия LLM: модель гарантированно выдаёт только валидные токены по JSON Schema/грамматике. Практически это значит: [\[acingai\]](#) [\[openreview\]](#)

- **Text-to-Pictiq транспайлер**: LLM переводит свободный текст в валидную последовательность tile-ID, используя ваш `icon-index.json` как грамматику/схему — то есть "сожми это сообщение до максимум 5 канонических тайлов по протоколу".
- **Безопасный ввод для киосков/embedded-интерфейсов**: если пользовательский ввод ограничен конечным набором тайлов вместо свободного текста, поверхность для prompt injection резко сокращается — это реальный аргумент для медицинских киосков, детских приложений, регулируемых сред, где вообще не хочется давать пользователю печатать произвольный текст в промт ИИ-ассистента.
- **Сжатие промпта / экономия токенов**: последовательность из 3–5 tile-ID может передавать intent устройства IoT в облачную LLM дешевле, чем полноценный текст — актуально для устройств с ограниченной связью.

Направление 3 — AAC + генеративный ИИ (самая горячая ниша прямо сейчас)

Это не гипотеза, а активно развивающееся исследовательское поле:

- **GenAAC** — система, которая генерирует AAC-символы динамически через LLM + text-to-image, решая проблему статичных баз символов. [\[pure.kaist.ac\]](#)
- **PrAACT** — предиктивная AAC на трансформерах, предсказывающая следующий символ/слово. [\[sciencedirect\]](#)

- Свежая работа по адаптации LLM под character-based AAC.[\[aclanthology\]](#)

Прямое применение к Pictiq: поскольку у вас уже формализована композиция (BASE+QUALIFIER, quantity, negation), можно обучить/настроить лёгкую предиктивную модель "какой следующий тайл вероятнее всего нужен пользователю", а генеративная часть — предлагать **новые кандидатные иконки на лету** для niche-концептов (по образцу GenAAC), которые потом проходят ваш формальный decision tree перед попаданием в канон. Это снимает главную боль AAC-систем — негибкость фиксированного словаря — но не жертвует вашей дисциплиной версионирования.

Направление 4 — ИИ-ИИ и человек-робот коммуникация

Здесь два разных, но связанных фронта:

- **Human-robot interaction (HRI)**: исследования explicitly показывают важность невербальных визуальных дисплеев для совместной работы человека и робота. Монохромный, компактный, спецификацией зафиксированный тайл — отличный кандидат на **e-ink/LED-дисплей робота или IoT-устройства** для статуса ("низкий заряд", "жду подтверждения", "ошибка") — не вместо голосового ассистента, а как минималистичный визуальный канал, который остаётся понятным без звука и на любом языке.[\[arxiv\]](#)
- **AI-AI / emergent communication**: в multi-agent RL агенты действительно спонтанно вырабатывают собственные символьные "языки" (AI Mother Tongue, arXiv 2507.10566; классическая работа Havrylov & Titov) — но у таких emergent-языков есть известная проблема: они непрозрачны для человека-наблюдателя. Здесь у Pictiq потенциально интересная роль — **интерпретируемый общий протокол-посредник**: если несколько разных ИИ-агентов (или агент + человек) должны координироваться в общей физической/визуальной среде, использование заранее зафиксированного, человекочитаемого символьного протокола вместо эмерджентного вектора эмбедингов даёт explainability "из коробки" — то есть человек в цикле всегда может прочитать, о чём "договариваются" агенты.[\[arxiv\]](#)[\[arxiv\]](#)

Направление 5 — data engineering: Pictiq как визуальный interlingua

Это, наверное, наименее эффектная, но наиболее быстро реализуемая идея — и её можно собрать буквально из уже существующих данных без всякого CV:

- Ваш `icon-index.json` с id + aliases + tags + examples на нескольких языках — это уже компактная **языконезависимая онтология понятий**, концептуально похожая на систему Q-идентификаторов Wikidata, только с человекочитаемым визуальным представлением вместо номера.

- Практический быстрый проект: сделать эмбединги по каждому тайлу (значение + алиасы + примеры), и через простой RAG-поиск сопоставлять произвольный текст (или распознанную речь) с ближайшей комбинацией тайлов — это уже сегодня даёт работающий прототип "текст → Pictiq-фраза" без обучения отдельной модели, только на основе существующего лексикона.
- Такой же слой можно использовать для **мультиязычной разметки датасетов**: если объект в изображении помечен tile-ID вместо слова на конкретном языке, разметка становится кросс-языковой по построению — потенциально полезно для обучающих датасетов в accessibility- и travel-tech продуктах.

Что реально стоит построить в первую очередь

1. **Формальная грамматика/JSON Schema протокола** — превращает уже написанные MUST/SHOULD правила в машинно-валидируемый артефакт, который можно скормить constrained-decoding библиотекам.
2. **RAG-прототип текст → tile-sequence** на основе существующего `icon-index.json` — недорого, без CV, демонстрирует главную идею за один уикенд разработки.
3. **Открытый бенчмарк "понимание Pictiq мультимодальными моделями"** по методологии, близкой к Visual Iconicity Challenge — исследовательский контент, который реально цитируем и продвигаем в AI-сообществе.
4. **MCP/agent-инструмент-переводчик** ("Pictiq translator tool"), который можно подключить к любому LLM-агенту как навык — низкая инвестиция, сразу открывает интеграцию с существующими ИИ-платформами.

Главная честная оговорка: конкурентное преимущество Pictiq в эпоху ИИ, скорее всего, не в том, чтобы "научить модели видеть картинки" — это быстро коммодитизируется современными VLM и без вас. Реальная ценность — в **закрытом, версионированном, человекочитаемом словаре и грамматике** как слое сжатия, безопасности ввода и объяснимости коммуникации между людьми и системами разного уровня понимания языка — именно то, чего нет ни у одного открытого эмбединг-пространства.

Архитектура: text → Pictiq-тайлы

Ключевая идея — не пытаться прогнать весь текст через один "чёрный ящик", а разделить работу так же, как это уже сделано в вашей собственной грамматике (`spec/GRAMMAR.md`): она различает **операторы** (закрытый, перечислимый класс — количество, отрицание, пунктуация) и **объекты** (открытый класс — `needs/places/items/...`). Отсюда — четыре стадии:

Stage A — Normalize & Segment. Свободный текст на любом языке → извлекается открытая концепция ("что нужно") отдельно от закрытых операторов (количество/отрицание/срочность/вопрос). В проде это один вызов LLM с constrained-decoding по

JSON Schema (та самая техника из прошлого ответа) — модель сразу возвращает структуру {concept, quantity, negation, urgent, question} на любом языке. В прототипе это заменено простыми регекс-правилами плюс маленьким демо-словарём RU→EN — намеренно тонкая заглушка, не часть архитектуры.

Stage B — Retrieve (реально работает). Строится корпус документов из `lexicon/icon-index.json`: каждая content-иконка (кроме операторных категорий) превращается в текст из `id + meaning + aliases` (с весом) + `tags`. TF-IDF по символьным n-граммам (не по словам — это важно, объясню ниже) ищет ближайшую иконку по косинусному сходству. Это офлайн, без внешних API, работает прямо на ваших реальных данных.

Stage C — Compose (реально работает). Найденный content-тайл + операторы собираются в последовательность по правилам из `GRAMMAR.md`: объект → количество → отрицание → пунктуация в конце.

Stage D — Validate / Fallback (реально работает). Если лучший score retrieval ниже порога — тайл не подставляется силой. Вместо этого фраза помечается как "нет уверенного совпадения" и указывается на `spec/STANDALONE_BACKLOG.md` — то есть отсутствие иконки становится структурированным сигналом для дизайна словаря, а не неверной галлюцинацией.

Что реально работает — прогон на демо-запросах

Вход	Тайлы	Score
Where can I find a toilet?	need_toilet + punct_question	0.275
I need two beers, please	drink_beer + qty_2	0.392
I need medical help urgently!	safety_medical + punct_exclaim	0.507
Мне нужна вода, срочно	need_water + punct_exclaim	0.501
Такси, пожалуйста?	move_taxi + punct_question	0.603
Where is the nearest hotel?	place_hotel + punct_question	0.348

Файл `pictiq_rag_prototype.py` в вашем воркспейсе — рабочий, я его запускал.

Два честных ограничения, найденных прямо при тестировании

- Составные предложения.** "No card, cash only" в одном проходе retrieval даёт только `money_coins + logic_no` — теряется вторая мысль (что карта не принимается, а наличные — да). Проверил вручную: если предварительно разбить фразу на два клоза ("no card" / "cash only") и прогнать каждый через `compose()` отдельно,

получается корректно: `money_card + logic_no` и `money_coins`. Это ровно то, ради чего Stage A в проде обязан быть настоящей LLM — сегментация многосмысловых фраз, а не regex.

2. **TF-IDF — не семантика, а буквенное сходство.** Запрос "I need sunscreen" (солнцезащитный крем — концепт, которого в лексиконе нет вообще) иногда всё равно даёт ложное совпадение с `need_bar` из-за случайного пересечения символьных n-грамм, а не смысла. Это ожидаемо для baseline без реальных эмбедингов и явно показывает, куда двигаться дальше.

Roadmap v2

1. Заменить TF-IDF на настоящие мультиязычные эмбединги (например multilingual-e5 или любую embedding-API) — уберёт и ложные совпадения по буквам, и необходимость в демо RU-словаре, потому что эмбединги понимают смысл, а не текстовое пересечение.
2. Реальная Stage A через LLM с function calling/JSON Schema — сразу решает и мультиязычность, и сегментацию составных фраз.
3. Прогон итоговой последовательности через decision tree из `PROTOCOL.md` перед выдачей — сейчас Stage D проверяет только "есть ли уверенное совпадение", но не проверяет валидность самой композиции по правилам грамматики.
4. Обернуть как инструмент для агентов (MCP/tool) — ровно тот "Pictiq translator", который обсуждали в прошлом ответе.